

UNIT – 4

8051 MICROCONTROLLER

Architecture - Addressing modes and Instruction Sets – Interrupt structure – Timer –I/O ports – Serial communication.

MICROCONTROLLERS VS MICROPROCESSORS

MICROPROCESSOR:

- ❖ A CPU built into a single VLSI chip is called a microprocessor.
- ❖ It is a general-purpose device and additional external circuitry is added to make it a microcomputer.
- ❖ The microprocessor contains arithmetic and logic unit (ALU), Instruction decoder and control unit, Instruction register, Program counter (PC), clock circuit (internal or external), reset circuit (internal or external) and registers.
- ❖ But the microprocessor has no on chip I/O Ports, Timers, Memory etc.
- ❖ For example, Intel 8085 is an 8-bit microprocessor and Intel 8086/8088 a 16-bit microprocessor.
- ❖ The block diagram of the Microprocessor is shown in Fig.4.1

MICROCONTROLLER:

- ❖ A microcontroller is a highly integrated single chip, which consists of on chip CPU (Central Processing Unit), RAM (Random Access Memory), EPROM/PROM/ROM (Erasable Programmable Read Only Memory), I/O (input/output) – serial and parallel, timers, interrupt controller.
- ❖ For example, Intel 8051 is 8-bit microcontroller and Intel 8096 is 16-bit microcontroller.
- ❖ The block diagram of Microcontroller is shown in Fig.4.2.

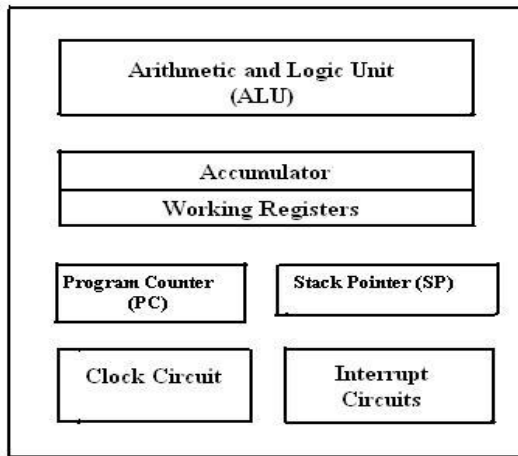


Fig.4.1 Block diagram of a Microprocessor

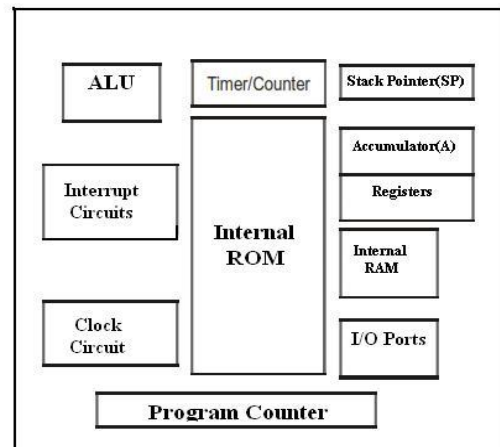


Fig.4.2 Block Diagram of a Microcontroller

DISTINGUISH BETWEEN MICROPROCESSOR AND MICROCONTROLLER:

Table 4.1 : Microprocessor Vs Microcontroller

S. No	Microprocessor	Microcontroller
1	A microprocessor is a general purpose device which is called a CPU	A microcontroller is a dedicated chip which is also called single chip computer.
2	A microprocessor do not contain on chip I/O Ports, Timers, Memories etc..	A microcontroller includes RAM, ROM, serial and parallel interface, timers, interrupt circuitry (in addition to CPU) in a single chip.
3	Microprocessors are most commonly used as the CPU in microcomputer systems	Microcontrollers are used in small, minimum component designs performing control-oriented applications.
4	Microprocessor instructions Are mainly nibble or byte addressable	Microcontroller instructions are both bit addressable as well as byte addressable.
5	Microprocessor instruction sets are mainly intended for catering to large volumes of data.	Microcontrollers have instruction sets catering to the control of inputs and outputs.
6	Microprocessor based system design is complex and expensive	Microcontroller based system design is rather simple and cost effective
7	The Instruction set of microprocessor is complex with large number of instructions.	The instruction set of a Microcontroller is very simple with less number of instructions. For, ex: PIC microcontrollers have only 35 instructions.
8	A microprocessor has zero status flag	A microcontroller has no zero flag.

The salient features of 8051Microcontroller are:

- ❖ 4 KB on chip program memory (ROM or EPROM).
- ❖ 128 bytes on chip data memory (RAM).
- ❖ 8-bit data bus
- ❖ 16-bit address bus
- ❖ 32 general purpose registers each of 8 bits
- ❖ Two -16 bit timers T0 and T1
- ❖ Five Interrupts (3 internal and 2 external).
- ❖ Four Parallel ports each of 8-bits (PORT0, PORT1, PORT2, PORT3) with a total of 32 I/O lines.
- ❖ One 16-bit program counter and One 16-bit DPTR (data pointer)
- ❖ One 8-bit stack pointer
- ❖ One Microsecond instruction cycle with 12 MHz Crystal.
- ❖ One full duplex serial communication port.

PIN DIAGRAM OF 8051:

- ❖ The 8051 microcontroller is available as a 40 pin DIP chip and it works at +5 volts DC.
- ❖ VCC → 5V supply
- ❖ VSS → GND
- ❖ XTAL2/XTAL1 are for oscillator input
- ❖ Port 0 – 32 to 39 – AD0/AD7 and P0.0 to P0.7
- ❖ Port 1 – 1 to 8 – P1.0 to P1.7
- ❖ Port 2 – 21 to 28 – P2.0 to P2.7 and A 8 to A15
- ❖ Port 3 – 10 to 17 – P3.0 to P3.7
- ❖ P 3.0 – RXD – Serial data input – SBUF
- ❖ P 3.1 – TXD – Serial data output – SBUF
- ❖ P 3.2 – INT0 – External interrupt 0 – TCON 0.1
- ❖ P 3.3 – INT1 – External interrupt 1 – TCON 0.3
- ❖ P 3.4 – T0 – External timer 0 input – TMOD

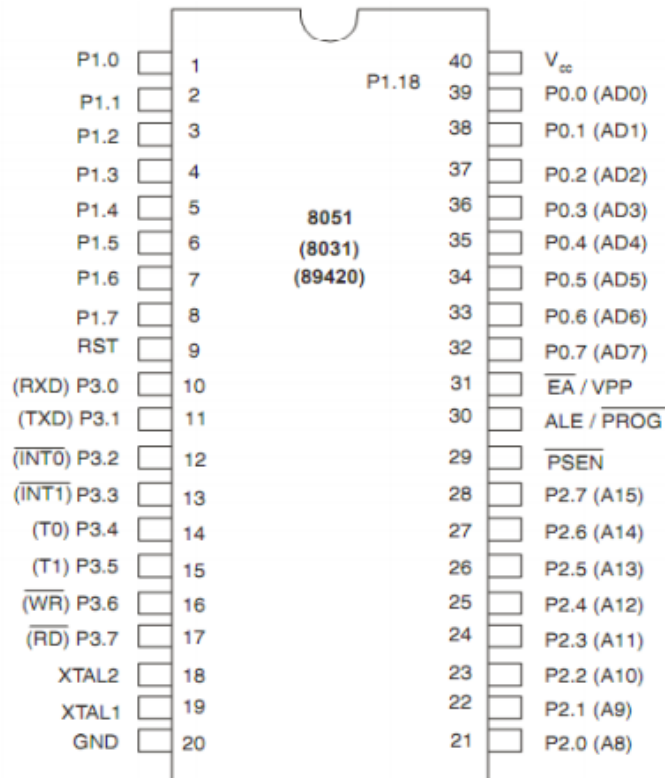
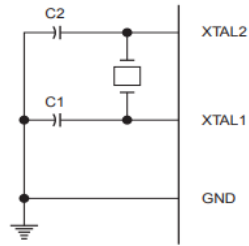


Fig 4.3 Pin Diagram of 8051

❖ XTAL1, XTAL2:

- These two pins are connected to Quartz crystal oscillator which runs the on-chip oscillator.
- The quartz crystal oscillator is connected to the two pins along with a capacitor of 30pF as shown in the figure 4.4.
- If we use a source other than the crystal oscillator, it will be connected to XTAL1 and XTAL2 is left unconnected as shown in figure 4.5.



Note: C1, C2 = 30 pF $\pm$ 10 pF for Crystals
= 40 pF $\pm$ 10 pF for Ceramic Resonators

Fig 4.4 Crystal Oscillator as Source

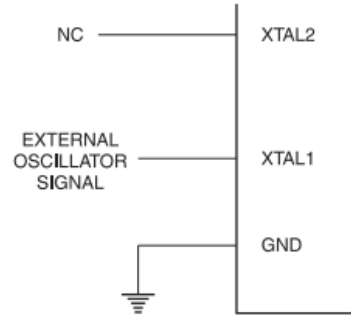


Fig 4.5 External Clock Source

❖ **P0.0- P0.7(AD0-AD7) :**

- The port 0 pins multiplexed with Address/data pins.
- If the microcontroller is accessing external memory these pins will act as address/data pins otherwise they are used for Port 0 pins.

❖ **P2.0- P2.7(A8-A15) :**

- The port2 pins are multiplexed with the higher order address pins.
- When the microcontroller is accessing external memory these pins provide the higher order address byte otherwise they act as Port 2 pins.

❖ **P1.0- P1.7:**

- These 8-pins are dedicated for Port1 to perform input or output port operations.

❖ **P3.0- P3.7:**

- These 8-pins are meant for Port3 operations and also for some control operations like Read, Write, Timer0, Timer1, INT0, INT1, Rx D and Tx D

❖ **RST:**

- The RESET pin is an input pin and it is an active high pin.
- When a high pulse is applied to this pin the microcontroller will reset and terminate all activities.
- Upon reset all the registers except PC will reset to 0000 Value and PC register will reset to 0007 value.

❖ **$\overline{EA}$ (External Access): Active Low Pin**

- This pin is connected to ground when microcontroller is accessing the program code stored in the external memory and connected to Vcc when it is accessing the program code in the on chip memory.
- This pin should not be left unconnected.

❖ **$\overline{PSEN}$ (Program Store Enable): Active Low Pin**

- When the microcontroller is accessing the program code stored in the external ROM, this pin is connected to the OE (Output Enable) pin of the ROM.

❖ **ALE (Address latch enable): Active High Pin**

- When connected to external memory, port 0 provides both address and data i.e address and data are multiplexed through port 0.
- This ALE pin will demultiplex the address and data bus.
- When the pin is High, the AD bus will act as address bus otherwise the AD bus will act as Data bus.

8051 ARCHITECTURE

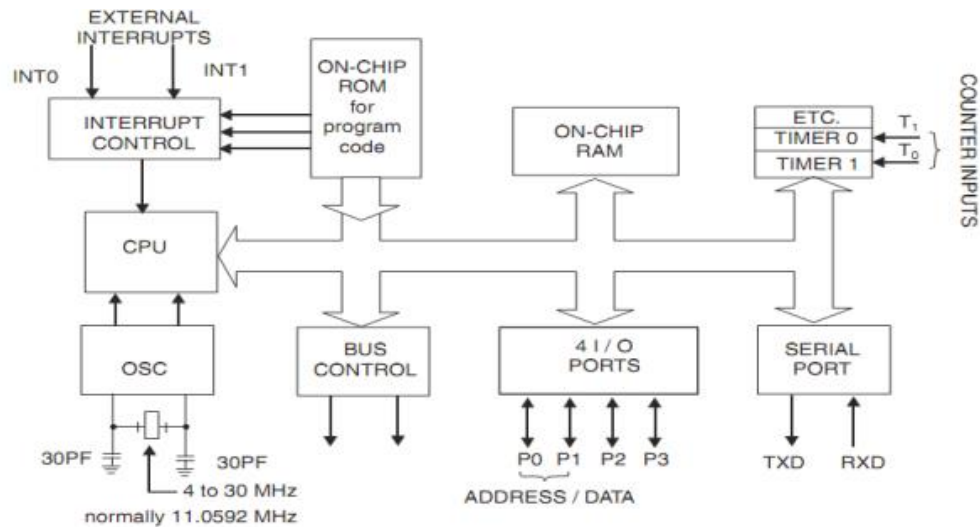


Fig 4.6 Block Diagram of 8051

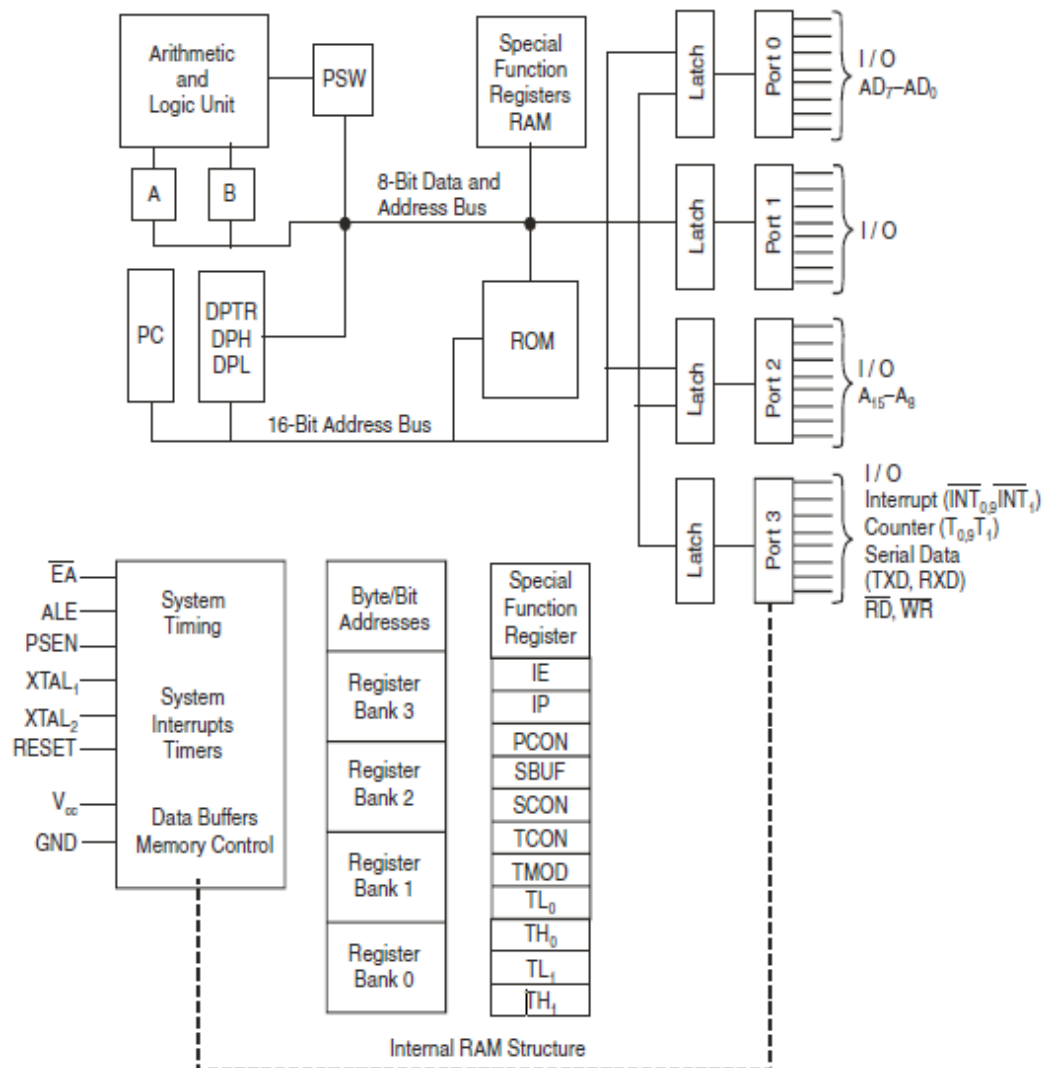


Fig 4.7 Architectural block diagram of microcontroller 8051

❖ Mostly used blocks in the architecture of 8051 are as follows:

❖ **ALU**

- Arithmetic Logical Unit
- This unit is used for the arithmetic calculations.

❖ **A-Accumulator**

- This register is used for arithmetic operations.
- This is also bit addressable and 8 bit register.

❖ **B-Register**

- This register is used in only two instructions MUL AB and DIV AB.
- This is also bit addressable and 8 bit register

❖ **PC-Program Counter**

- Points to the address of next instruction to be executed from ROM
- It is 16 bit register means the 8051 can access program address from 0000H to FFFFH. A total of 64KB of code.
- Initially PC has 0000H
- ORG instruction is used to initialize the PC
 - ORG 0000H means PC initialize by 0000H
- PC is incremented after each instruction. ROM

❖ **PSW Register (8051 Flags)**

- Used to indicate the Arithmetic condition of Accumulator.
- Flag register in 8051 is called as program status word (PSW).
- This special function register PSW is also bit addressable and 8 bit wide means each bit can be set or reset independently.

PSW0.7	PSW0.6	PSW0.5	PSW0.4	PSW0.3	PSW0.2	PSW0.1	PSW0.0
CY	AC	F0	RS1	RS0	OV	—	P

FLAG Register

- The meaning of various bits of PSW register is shown below.

Table 4.2: PSW Register Description

CY	PSW0.7	Carry Flag
AC	PSW0.6	Auxiliary Carry Flag
F0	PSW0.5	Flag 0 available for general purpose.
RS1	PSW0.4	Register Bank select bit 1
RS0	PSW0.3	Register bank select bit 0
OV	PSW0.2	Overflow flag
---	PSW0.1	User definable flag
P	PSW0.0	Parity flag .set/cleared by hardware.

- The bits PSW3 and PSW4 are denoted as RS0 and RS1 and these bits are used to select the bank registers of the RAM location.
- **P (Parity flag)** → PSW 0.0
 - 1 – odd number of 1 in ACC
 - 0 – even number of 1 in ACC

- **OV (overflow flag)** → PSW 0.2
 - This is used to detect error in signed arithmetic operation.
 - This is similar to carry flag but difference is only that carry flag is used for unsigned operation.
- **The selection of the register Banks** and their addresses are given below.

Table 4.3: Register Bank Selection

RS1	RS0	Register Bank	Address
0	0	0	00H-07H
0	1	1	08H-0FH
1	0	2	10H-17H
1	1	3	18H-1FH

- Initially by default always Bank 0 is selected.
- **F0 → user definable bit**
- **AC (Auxiliary carry flag)** → PSW0.6
 - When carry is generated from D3 to D4, it is set to 1, it is used in BCD arithmetic.
- **CY (carry flag)** → PSW0.7
 - Affected after 8 bit addition and subtraction.
 - It is used to detect error in unsigned arithmetic operation.
 - It can be used as single bit storage.
 - SETB C → for cy = 1
 - CLR C → for cy = 0

❖ **Stack in 8051**

- RAM locations from 08H to 1FH can be used as stack. Stack is used to store the data temporarily.
- Stack is last in first out (LIFO)
- Stack pointer (SP)
 - 8bit register
 - It indicates current RAM address available for stack or it points the top of stack.
 - Initially by default at 07H because first location of stack is 08H.
 - After each PUSH instruction the SP is incremented by one while in MC after PUSH instruction SP is decremented.
 - After each POP instruction the SP is decremented.

❖ **DPTR (Data Pointer) in 8051**

- This is a 16 bit register which is used to furnish address information for internal and external program memory and for external data memory.
- It is divided into two parts DPH and DPL
 - DPH for Higher order 8 bits, DPL for lower order

❖ **Input Output Ports**

- There are four input output ports available P0, P1, P2, P3.

- Each port is 8 bit wide and has special function register P0, P1, P2, P3 which are bit addressable means each bit can be set or reset by the Bit instructions (SETB for high, CLR for low) independently.
- The data at any port which is transmitting or receiving is in these registers.
- The port 0 can perform dual works. It is also used as Lower order address bus (A0 to A7) multiplexed with 8 bit data bus P0.0 to P0.7 is AD0 to AD7 respectively the address bus and data bus is demultiplexed by the ALE signal and latch which is further discussed in details.
- Port 2 can be used as I/O port as well as higher order address bus A8 to A15.
- Port 3 also have dual functions it can be worked as I/O as well as each pin of P3 has specific function.
- P3.0 – RXD – {Serial I / P for Asynchronous communication Serial O / P for synchronous communication}.
- P3.1 – TXD – Serial data transmit.
- P3.2 – INT0 – External Interrupt 0.
- P3.3 – INT1 – External Interrupt 1.
- P3.4 – T0 – Clock input for counter 0.
- P3.5 – T1 – Clock input for counter 1.
- P3.6 – WR – Signal for writing to external memory.
- P3.7 – RD – Signal for reading from external memory.
- When external memory is interfaced with 8051 then P0 and P2 can't be worked as I/O port they works as address bus and data bus, otherwise they can be accessed as I/O ports.

❖ Oscillator

- It is used for providing the clock to MC8051 which decides the speed or baud rate of MC.
- We use crystal which frequency varies from 4MHz to 30 MHz, normally we use 11.0592 MHz frequency.

❖ Interrupts

- Interrupts are defined as requests because they can be refused (masked) if they are not used, that is when an interrupt is acknowledged.
- A special set of events or routines are followed to handle the interrupts.
- These special routines are known as interrupt handler or interrupt service routines (ISR).
- These are located at a special location in memory.
- INT0 and INT1 are the pins for external interrupts.

❖ Timers and Counters

- Timer means which can give the delay of particular time between some events. For example on or off the lights after every 2 sec.
- This delay can be provided through some assembly program but in microcontroller two hardware pins are available for delay generation.
- These hardware pins can be also used for counting some external events.
- How much time a number is repeated in the given table is calculated by the counter?

- In MC8051, two timer pins are available T0 and T1, by these timers we can give the delay of particular time if we use these in timer mode.
- We can count external pulses at these pins if we use these pins in counter mode.
- 16 bits timers are available. Means we can generate delay between 0000H to FFFFH.
- Two special function registers are available.



- If we want to load T0 with 16 bit data then we can load separate lower 8 bit in TL0 and higher 8 bit in TH0.
- In the same way for T1.
- TMOD, TCON registers are used for controlling timer operation.

❖ **Serial Port**

- There are two pins available for serial communication TXD and RXD
- Normally TXD is used for transmitting serial data which is in SBUF register, RXD is used for receiving the serial data.
- SCON register is used for controlling the operation.

❖ **Memory organization:**

- The 8051 microcontroller has 128 bytes of Internal RAM and 4kB of on chip ROM.
- The RAM is also known as Data memory and the ROM is known as program memory.
- The program memory is also known as Code memory.
- This Code memory holds the actual 8051 program that is to be executed.
- In 8051 this memory is limited to 64K.
- Code memory may be found on-chip, as ROM or EPROM. It may also be stored completely off-chip in an external ROM or, more commonly, an external EPROM.
- The 8051 has only 128 bytes of Internal RAM but it supports 64kB of external RAM. As the name suggests, external RAM is any random access memory which is off-chip.
- Since the memory is off-chip it is not as flexible in terms of accessing, and is also slower.
- For example, to increment an Internal RAM location by 1, it requires only 1 instruction and 1 instruction cycle but to increment a 1-byte value stored in External RAM requires 4 instructions and 7 instruction cycles.
- So, here the external memory is 7 times slower

❖ **Internal RAM OF 8051:**

- This Internal RAM is found on-chip on the 8051.
- So it is the fastest RAM available, and it is also the most flexible in terms of reading, writing, and modifying its contents.

- Internal RAM is volatile, so when the 8051 is reset this memory is cleared.
- The 128 bytes of internal
 - Four register banks (Bank0, Bank1, Bank2 and Bank3) each of 8-bits (total 32 bytes). The default bank register is Bank0. The remaining Banks are selected with the help of RS0 and RS1 bits of PSW Register.
 - 16 bytes of bit addressable area and
 - 80 bytes of general purpose area (Scratch pad memory) as shown in the figure 4.8. This area is also utilized by the microcontroller as a storage area for the operating stack.
- The 32 bytes of RAM from address 00 H to 1FH are used as working registers organized as four banks of eight registers each.
- The registers are named as R0-R7 .Each register can be addressed by its name or by its RAM address.

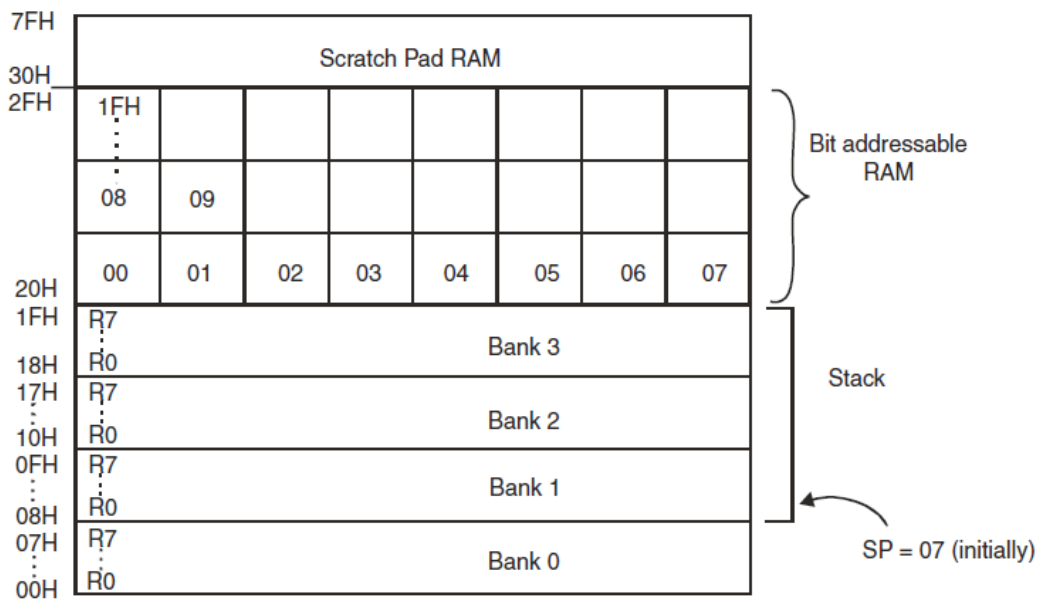


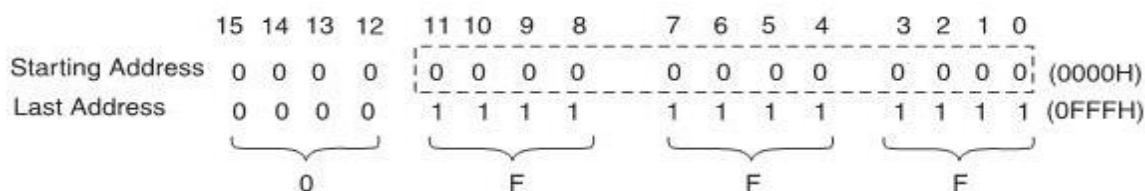
Fig 4.8 Structure of RAM

❖ On chip ROM

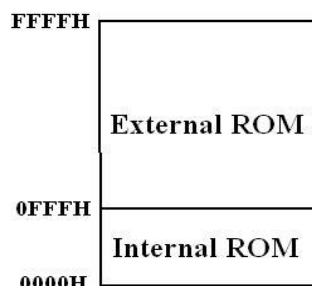
- In 8051, 4KB read only memory (ROM) is available for program storage.
- This is used for permanent data storage. Or the data which is not changed during the processing like the program or algorithm for specific applications.
 - This is volatile memory; the data saved in this memory does not disappear after power failure.
 - We can interface up to 64KB ROM memory externally if the application is large.
- These sizes are specified different by their companies.
- Address Range of PC: Address range of PC means program counter (which points the next instruction to be executing) can be moved between these locations or we can save the program from this location to this location. The address range can be calculated in the same way just like the RAM which is discussed in previous section

$$4\text{KB} = 2^2 \cdot 2^{10} \text{ B (since } 1\text{KB} = 2^{10} \text{ B)}$$

$$= 2^{12} \text{ Byte}$$



- Address range of PC is 0000H to 0FFFH means total 4KB locations are available from 0000H to 0FFFH.



❖ SPECIAL FUNCTION REGISTERS (SFRs):

- In 8051 microcontroller there certain registers which uses the RAM addresses from 80h to FFh and they are meant for certain specific operations.
- These registers are called Special function registers (SFRs).
- Some of these registers are bit addressable also.
- The list of SFRs and their functional names are given below.
- In these SFRs some of them are related to I/O ports (P0, P1, P2 and P3) and some of them are meant for control operations (TCON, SCON, PCON..) and remaining are the auxiliary SFRs, in the sense that they don't directly configure the 8051.

Table 4.4: SFRs of 8051 Microcontroller

S.No	Symbol	Name of SFR	Address (Hex)
1	ACC*	Accumulator	E0
2	B*	B-Register	F0
3	PSW*	Program Status word register	D0
4	SP	Stack Pointer Register	81
5	DPTR	DPL	82
		DPH	83
6	P0*	Port 0	80
7	P1*	Port 1	90
8	P2*	Port 2	0A
9	P3*	Port 3	0B
10	IP*	Interrupt Priority control	B8
11	IE*	Interrupt Enable control	A8

12	TMOD	Timer mode register	89
13	TCON*	Timer control register	88
14	TH0	Timer 0 Higher byte	8C
15	TL0	Timer 0 Lower byte	8A
16	TH1	Timer 1 Higher byte	8D
17	TL1	Timer 1 lower byte	8B
18	SCON*	Serial control register	98
19	SBUF	Serial buffer register	99
20	PCON	Power control register	87

The * indicates the bit addressable SFRs

ADDRESSING MODES OF 8051:

- ❖ The way in which the data operands are accessed by different instructions is known as the addressing modes.
- ❖ There are various methods of denoting the data operands in the instruction.
- ❖ The 8051 microcontroller supports mainly 5 addressing modes.
- ❖ They are
 - Immediate addressing mode
 - Direct Addressing mode
 - Register addressing mode
 - Register Indirect addressing mode
 - Indexed addressing mode
- ❖ **Immediate addressing mode:**
 - The addressing mode in which the data operand is a constant and it is a part of the instruction itself is known as Immediate addressing mode. Normally the data must be preceded by a # sign. This addressing mode can be used to transfer the data into any of the registers including DPTR.
 - **Ex:**
 - MOV A, # 27 H:** The data (constant) 27 is moved to the accumulator register
 - ADD R1, #45 H:** Add the constant 45 to the contents of the accumulator
 - MOV DPTR, # 8245H:** Move the data 8245 into the data pointer register.
 - MOV P1, #21H:** Move the data 21 to PORT1
- ❖ **Direct addressing mode:**
 - The addressing mode in which the data operand is in the RAM location (00 -7FH) and the address of the data operand is given in the instruction is known as Direct addressing mode. The direct addressing mode uses the lower 128 bytes of Internal RAM and the SFRs
 - **Ex**
 - MOV R1, 42H:** Move the contents of RAM location 42 into R1 register
 - MOV 49H, A:** Move the contents of the accumulator into the RAM location 49.
 - ADD A, 56H:** Add the contents of the RAM location 56 to the accumulator
- ❖ **Register addressing mode:**

- The addressing mode in which the data operand to be manipulated lies in one of the registers is known as register addressing mode.
- **Ex**
MOV A, R0: Move the contents of the register R0 to the accumulator
ADD A, R6: Add the contents of R6 register to the accumulator
MOV P1, R2: Move the contents of the R2 register into port 1
MOV R5, R2: This is invalid .The data transfer between the registers is not allowed.
- ❖ **Register indirect addressing mode:**
 - The addressing mode in which a register is used as a pointer to the data memory block is known as Register indirect addressing mode.
 - **Ex**
MOV A, @ R0: Move the contents of RAM location whose address is in R0 into A (accumulator)
MOV @ R1, B: Move the contents of B into RAM location whose address is held by R1 When R0 and R1 are used as pointers, they must be preceded by @ sign
 - **One of the advantages of register indirect addressing mode is that it makes accessing the data more dynamic than static as in the case of direct addressing mode.**
- ❖ **Indexed addressing mode:**
 - This addressing mode is used in accessing the data elements of lookup table entries located in program ROM space of 8051.
 - **Ex:**
MOVC A,@ A+DPTR
The 16-bit register DPTR and register A are used to form the address of the data element stored in on-chip ROM. Here C denotes code. In this instruction the contents of An are added to the 16-bit DPTR register to form the 16-bit address of the data operand

INSTRUCTION SETS:

- ❖ The instructions of 8051 can be broadly classified under the following headings.
 1. Data transfer Instructions
 2. Arithmetic Instructions
 3. Logical Instructions
 4. Program Branching Instructions
 5. Bit Manipulation Instructions / Boolean Variable Manipulation Instructions
- ❖ **Arithmetic Operations**
 - **ADD A,Rn →** Add register to Accumulator
 - **ADD A,direct →** Add direct byte to Accumulator
 - **ADD A,@Ri →** Add indirect RAM to Accumulator
 - **ADD A,#data →** Add immediate data to Accumulator
 - **ADDC A,Rn →** Add register to Accumulator with Carry
 - **ADDC A,direct→** Add direct byte to Accumulator with Carry

- ADDC A,@Ri → Add indirect RAM to Accumulator with Carry
- ADDC A,#data → Add immediate data to Acc with Carry
- SUBB A,Rn → Subtract Register from Acc with borrow
- SUBB A,direct → Subtract direct byte from Acc with borrow
- SUBB A,@Ri → Subtract indirect RAM from ACC with borrow
- SUBB A,#data → Subtract immediate data from Acc with borrow
- INC A → Increment Accumulator
- INC Rn → Increment register
- INC direct → Increment direct byte
- INC @Ri → Increment direct RAM
- DEC A → Decrement Accumulator
- DEC Rn → Decrement Register
- DEC direct → Decrement direct byte
- DEC @Ri → Decrement indirect RAM
- INC DPTR → Increment Data Pointer
- MUL AB → Multiply A & B
- DIV AB → Divide A by B
- DA A → Decimal Adjust Accumulator

❖ **LOGICAL OPERATIONS**

- ANL A,Rn → AND Register to Accumulator
- ANL A,direct → AND direct byte to Accumulator
- ANL A,@Ri → AND indirect RAM to Accumulator
- ANL A,#data → AND immediate data to Accumulator
- ANL direct,A → AND Accumulator to direct byte
- ANL direct,#data → AND immediate data to direct byte
- ORL A,Rn → OR register to Accumulator
- ORL A,direct → OR direct byte to Accumulator
- ORL A,@Ri → OR indirect RAM to Accumulator
- ORL A,#data → OR immediate data to Accumulator
- ORL direct,A → OR Accumulator to direct byte
- ORL direct,#data → OR immediate data to direct byte
- XRL A,Rn → Exclusive-OR register to Accumulator
- XRL A,direct → Exclusive-OR direct byte to Accumulator
- XRL A,@Ri → Exclusive-OR indirect RAM to Accumulator
- XRL A,#data → Exclusive-OR immediate data to Accumulator
- XRL direct,A → Exclusive-OR Accumulator to direct byte
- XRL direct,#data → Exclusive-OR immediate data to direct byte
- CLR A → Clear Accumulator
- CPL A → Complement Accumulator
- RL A → Rotate Accumulator Left
- RLC A → Rotate Accumulator Left through the Carry
- RR A → Rotate Accumulator Right
- RRC A → Rotate Accumulator Right through the Carry

- SWAP A → Swap nibbles within the Accumulator

❖ **DATA TRANSFER**

- MOV A,Rn → Move register to Accumulator
- MOV A,direct → Move direct byte to Accumulator
- MOV A,@Ri → Move indirect RAM to Accumulator
- MOV A,#data → Move immediate data to Accumulator
- MOV Rn,A → Move Accumulator to register
- MOV Rn,direct → Move direct byte to register
- MOV Rn,#data → Move immediate data to register
- MOV direct,A → Move Accumulator to direct byte
- MOV direct,Rn → Move register to direct byte
- MOV direct,direct → Move direct byte to direct
- MOV direct,@Ri → Move indirect RAM to direct byte
- MOV direct,#data → Move immediate data to direct byte
- MOV @Ri,A → Move Accumulator to indirect RAM
- MOV @Ri,direct → Move direct byte to indirect RAM
- MOV @Ri,#data → Move immediate data to indirect RAM
- MOV DPTR,#data16 → Load Data Pointer with a 16-bit constant
- MOVC A,@A+DPTR → Move Code byte relative to DPTR to Acc
- MOVC A,@A+PC → Move Code byte relative to PC to Acc
- MOVX A,@Ri → Move External RAM (8-bit addr) to Acc
- MOVX A,@DPTR → Move External RAM (16-bit addr) to Acc
- MOVX @Ri,A → Move Acc to External RAM (8-bit addr)
- MOVX @DPTR,A → Move Acc to External RAM (16-bit addr)
- PUSH direct → Push direct byte onto stack
- POP direct → Pop direct byte from stack
- XCH A,Rn → Exchange register with Accumulator
- XCH A,direct → Exchange direct byte with Accumulator
- XCH A,@Ri → Exchange indirect RAM with Accumulator
- XCHD A,@Ri → Exchange low-order Digit indirect RAM with Acc

❖ **BOOLEAN VARIABLE MANIPULATION**

- CLR C → Clear Carry
- CLR bit → Clear direct bit
- SETB C → Set Carry
- SETB bit → Set direct bit
- CPL C → Complement Carry
- CPL bit → Complement direct bit
- ANL C,bit → AND direct bit to CARRY
- ANL C,/bit → AND complement of direct bit to Carry
- ORL C,bit → OR direct bit to Carry
- ORL C,/bit → OR complement of direct bit to Carry
- MOV C,bit → Move direct bit to Carry

- MOV bit,C →Move Carry to direct bit
- JC rel →Jump if Carry is set
- JNC rel→ Jump if Carry not set
- JB bit,rel→ Jump if direct Bit is set
- JNB bit,rel →Jump if direct Bit is Not set
- JBC bit,rel →Jump if direct Bit is set & clear bit

❖ **PROGRAM BRANCHING**

- ACALL addr11→ Absolute Subroutine Call
- LCALL addr16→Long Subroutine Call
- RET →Return from Subroutine
- RETI →Return from interrupt
- AJMP addr11→ Absolute Jump
- LJMP addr16 →Long Jump
- SJMP rel →Short Jump (relative addr)
- JMP @A+DPTR→ Jump indirect relative to the DPTR
- JZ rel →Jump if Accumulator is Zero
- JNZ rel→ Jump if Accumulator is Not Zero
- CJNE A,direct,rel →Compare direct byte to Acc and Jump if Not Equal
- CJNE A,#data,rel →Compare immediate to Acc and Jump if Not Equal
- CJNE Rn,#data,rel →Compare immediate to register and Jump if Not Equal
- CJNE @Ri,#data,rel →Compare immediate to indirect and Jump if Not Equal
- DJNZ Rn,rel →Decrement register and Jump if Not Zero
- DJNZ direct,rel →Decrement direct byte and Jump if Not Zero
- NOP →No Operation

Table 4.5: JUMP Instructions Range

Instructions	Jumping Range	
JZ, JNZ, DJNZ, CJNE, JC, JNC, JB, JNB, JBC	-128 to +128 bytes of the contents in PC	Conditional Jumps
LJMP	0000 to FFFF memory Location	Un-Conditional Jumps
SJMP	128 to +128 bytes of the contents in PC	
LCALL	Within 64KB Range	CALL Instructions
ACALL	Within 2KB Range	

PROGRAMS:

PROGRAM FOR 8-BIT ADDITION USING 8051 MICROCONTROLLER				
ADDRESS	LABEL	OPCODE	OPERAND	COMMENTS
9000		MOV A,	#80H	MOVE THE VALUE TO A REGISTER
9001				
9002		MOV R0,	#90H	MOVE THE VALUE TO R0 REGISTER
9003				

9004		MOV R1,	#00H	CLEAR THE R1 REGISTER
9005				
9006		ADD A,R0		ADD TWO 8 BIT VALUES
9007		JNC	900A	JUMP ON NO CARRY TO SPECIFIED LOCATION
9008				
9009		INC R1		INCREMENT R1 REGISTER
900A		MOV DPTR,	#9500	MOVE DPTR TO SPECIFIED ADDRESS LOCATION
900B				
900C				
900D		MOVX @DPTR,A		MOVE THE A REGISTER VALUE TO DPTR
900E		INC DPTR		INCREMENT DPTR
900F		MOV A,R1		MOVE THE R1 REGISTER VALUE TO A
9010		MOVX @DPTR,A		MOVE THE A REGISTER VALUE TO ADDRESS OF DPTR
9011	XXX	SJMP	9011	SHORT JUMP TO SPECIFIED ADDRESS
9012			XXX	
9013				

PROGRAM FOR 8-BIT SUBTRACTION USING 8051 MICROCONTROLLER

ADDRESS	LABEL	OPCODE	OPERAND	COMMENTS
9000		MOV A,	#FFH	MOVE THE VALUE TO A REGISTER
9001				
9002		MOV R0,	#80H	MOVE THE VALUE TO R0 REGISTER
9003				
9004		MOV R1,	#00H	CLEAR THE R1 REGISTER
9005				
9006		SUBB A,R0		SUBTRACT TWO 8 BIT VALUES
9007		JNC	900C	JUMP ON NO CARRY TO SPECIFIED LOCATION
9008				
9009		CPL A		COMPLEMENT THE ACCUMULATOR
900A		INC A		INCREMENT A REGISTER
900B		INC R1		INCREMENT R1 REGISTER
900C		MOV DPTR,	#9500	MOVE THE DPTR TO SPECIFIED ADDRESS
900D				
900E				
900F		MOVX @DPTR,A		MOVE A REGISTER TO ADDRESS OF

				DPTR
9010		INC DPTR		INCREMENT DPTR VALUE
9011		MOV A,R1		MOVE THE R1 REGISTER TO A REGISTER
9012		MOVX @DPTR,A		MOVE A REGISTER TO ADDRESS OF DPTR
9013	XXX	SJMP	9011	SHORT JUMP TO SPECIFIED ADDRESS
9014			XXX	
9015				

PROGRAM FOR 8-BIT MULTIPLICATION USING 8051 MICROCONTROLLER

ADDRESS	LABEL	OPCODE	OPERAND	COMMENTS
9000		MOV DPTR,	#9500	MOVE DPTR TO SPECIFIED ADDRESS
9001				
9002				
9003		MOVX A,@DPTR		MOVE THE ADDRESS OF DPTR TO A REGISTER
9004		MOV F0,A		MOVE A REGISTER TO F0
9005		INC DPTR		INCREMENT DPTR
9006		MOVX A,@DPTR		MOVE THE ADDRESS OF DPTR TO A REGISTER
9007		MUL AB		MULTIPLY 8 BIT VALUE
9008		INC DPTR		INCREMENT DPTR
9009		MOVX @DPTR,A		MOVE A REGISTER TO ADDRESS OF DPTR
900A		INC DPTR		INCREMENT DPTR
900B		MOV A,F0		MOVE VALUE FROM F0 TO A
900C		MOVX @DPTR,A		MOVE A REGISTER TO ADDRESS OF DPTR
900D	XXX	<u>SJMP</u>	900D	SHORT JUMP TO SPECIFIED LOCATION
900E			XXX	
900F				

PROGRAM FOR 8-BIT DIVISION USING 8051 MICROCONTROLLER

ADDRESS	LABEL	OPCODE	OPERAND	COMMENTS
9000		MOV DPTR,	#9500	MOVE DPTR TO SPECIFIED ADDRESS
9001				
9002				
9003		MOVX A,@DPTR		MOVE THE ADDRESS OF DPTR TO A REGISTER

9004		MOV F0,A		MOVE A REGISTER TO F0
9005		INC DPTR		INCREMENT DPTR
9006		MOVX A,@DPTR		MOVE THE ADDRESS OF DPTR TO A REGISTER
9007		DIV AB		DIVIDE 8 BIT VALUES
9008		INC DPTR		INCREMENT DPTR
9009		MOVX @DPTR,A		MOVE A REGISTER TO ADDRESS OF DPTR
900A		INC DPTR		INCREMENT DPTR
900B		MOV A,F0		MOVE VALUE FROM F0 TO A
900C		MOVX @DPTR,A		MOVE A REGISTER TO ADDRESS OF DPTR
900D	XXX	<u>SJMP</u>	900D	SHORT JUMP TO SPECIFIED LOCATION
900E			XXX	
900F				

Note 1: In Multiplication,

- ❖ A holds MSB of the result and B holds LSB of the result
- ❖ This instruction always makes CY and OV flag as '0', if result is less than 'FF'
- ❖ If CY=0 and OV=1, indicates the result is above 'FF'

Note 2: In division,

- ❖ A holds Quotient and B holds Remainder
- ❖ This instruction always makes CY and OV flag as '0', if denominator is not '0'
- ❖ If CY=0 and OV=1, indicates an error that denominator is zero

PARALLEL I/O PORTS:

- ❖ The 8051 microcontroller has four parallel I/O ports, each of 8-bits.
- ❖ So, it provides the user 32 I/O lines for connecting the microcontroller to the peripherals.
- ❖ The four ports are P0 (Port 0), P1(Port1) ,P2(Port 2) and P3 (Port3).
- ❖ Upon **reset all the ports are output ports.**
- ❖ In order to make them **input, all the ports must be set** i.e a high bit must be sent to all the port pins.
- ❖ This is normally done by the instruction "SETB".

Ex:

MOV A,#0FFH; A = FF

MOV P0,A; make P0 an input port

PORT 0:

- ❖ Port 0 is an 8-bit I/O port with dual purpose.
- ❖ If external memory is used, these port pins are used for the lower address byte address/data (AD0-AD7), otherwise all bits of the port are either input or output.

- ❖ Unlike other ports, Port 0 is **not provided with pull-up resistors internally**, so for PORT0 pull-up resistors of nearly 10k are to be connected externally as shown in the fig.4.10.

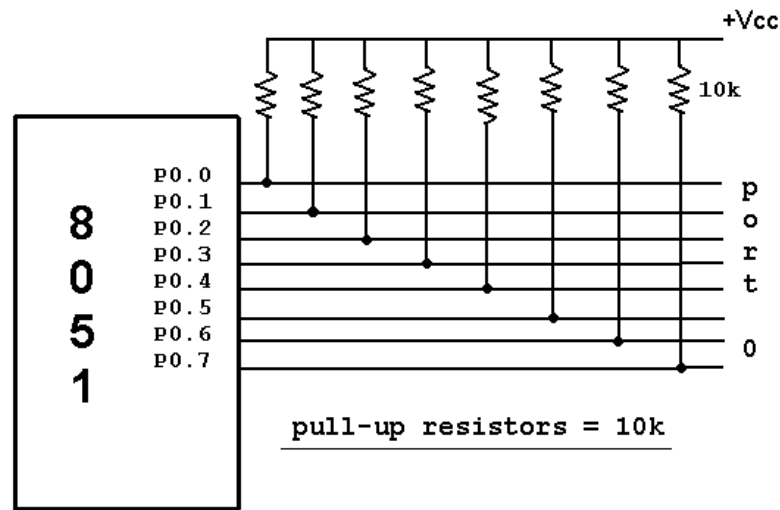


Fig 4.10 PORT 0 With Pull-Up Resistor Connection

Dual role of port 0:

- ❖ Port 0 can also be used as address/data bus (AD0-AD7), allowing it to be used for both address and data.
- ❖ When connecting the 8051 to an external memory, port 0 provides both address and data.
- ❖ The 8051 multiplexes address and data through port 0 to save the pins.
- ❖ ALE indicates whether P0 has address or data.
- ❖ When ALE = 0, it provides data D0-D7, and when ALE =1 it provides address and data with the help of a 74LS373 latch

Port 1:

- ❖ Port 1 occupies a total of 8 pins (pins 1 through 8).
- ❖ It has no dual application and acts only as input or output port.
- ❖ In contrast to port 0, this port does not need any pull-up resistors since pull-up resistors connected internally.
- ❖ Upon reset, Port 1 is configured as an output port.
- ❖ To configure it as an input port, port bits must be set i.e a high bit must be sent to all the port pins.
- ❖ This is normally done by the instruction "SETB".

Ex:

MOV A, #0FFH; A=FF HEX

MOV P1, A; make P1 an input port by writing 1's to all of its pins

Port 2:

- ❖ Port 2 is also an eight bit parallel port. (pins 21- 28).
- ❖ It can be used as input or output port.

- ❖ As this port is provided with internal pull-up resistors it does not need any external pull-up resistors.
- ❖ Upon reset, Port 2 is configured as an output port.
- ❖ If the port is to be used as input port, all the port bits must be made high by sending FF to the port.

Ex:

MOV A, #0FFH; A=FF hex

MOV P2, A; make P2 an input port by writing all 1's to it

Dual role of port 2:

- ❖ Port2 lines are also associated with the higher order address lines A8-A15.
- ❖ In systems based on the 8751, 8951, and DS5000, Port2 is used as simple I/O port.
- ❖ But, in 8051/31-based systems, port 2 is used along with P0 to provide the 16-bit address for the external memory.
- ❖ Since an 8051/31 is capable of accessing 64K bytes of external memory, it needs a path for the 16 bits of the address.
- ❖ While P0 provides the lower 8 bits via A0-A7, it is the job of P2 to provide bits A8-A15 of the address.
- ❖ In other words, when 8031 is connected to external memory, Port 2 is used for the upper 8 bits of the 16 bit address, and it cannot be used for I/O operations.

PORT 3:

- ❖ Port3 is also an 8-bit parallel port with dual function.(pins 10 to 17).
- ❖ The port pins can be used for I/O operations as well as for control operations.
- ❖ The details of these additional operations are given below in the table.
- ❖ Port 3 also do not need any external pull-up resistors as they are provided internally similar to the case of Port2 & Port 1.
- ❖ Upon reset port 3 is configured as an output port .
- ❖ If the port is to be used as input port, all the port bits must be made high by sending FF to the port.

For ex,

MOV A, #0FFH; A= FF hex

MOV P3, A; make P3 an input port by writing all 1's to it

Alternate Functions of Port 3:

- ❖ P3.0 and P3.1 are used for the RxD (Receive Data) and TxD (Transmit Data) serial communications signals.
- ❖ Bits P3.2 and P3.3 are meant for external interrupts.
- ❖ Bits P3.4 and P3.5 are used for Timers 0 and 1 and P3.6 and P3.7 are used to provide the write and read signals of external memories connected in 8031 based systems.
 - P3.1 – TXD – Serial data transmit.
 - P3.2 – INT0 – External Interrupt 0.
 - P3.3 – INT1 – External Interrupt 1.
 - P3.4 – T0 – Clock input for counter 0.
 - P3.5 – T1 – Clock input for counter 1.
 - P3.6 – WR – Signal for writing to external memory.

- P3.7 – RD – Signal for reading from external memory.

Programming I/O Ports:

- ❖ Write a test program for the 8051 chip to toggle all the bits of P0,P1 and P2 after a delay.

```

MOV P0,#00
MOV P1,#00
MOV P2,#00
BACK: MOV A,#55
      MOV P0,A
      MOV P1,A
      MOV P2,A
      ACALL DELAY
      MOV A,#AA
      MOV P0,A
      MOV P1,A
      MOV P2,A
      ACALL DELAY
      SJMP BACK

```

```

DELAY:MOV R1,#FF
J1:    DJNZ R1,J1
      RET

```

- ❖ Write a program to generate a square waveform from bit0 of port1.

```

BACK: SETB P1.0
      LCALL DELAY
      CLR P1.0
      LCALL DELAY
      SJMP BACK

```

```

DELAY:MOV R1,#FF
J1:    DJNZ R1,J1
      RET

```

OR

```

BACK: CPL P1.0
      LCALL DELAY
      SJMP BACK

```

- ❖ A switch is connected to pin P1.7 and an LED to pin P2.0. Write a program to get the status of the switch and send it to the LED.

```

LED BIT P1.7
SW BIT P2.0
HERE: MOV C, SW
      MOV LED, C
      SJMP HERE

```

- ❖ Assume that bit P2.3 is an input and represents the condition of an oven. If it goes high, it means that the oven is hot. Monitor the bit continuously. Whenever it goes high, send a high-to-low pulse to port P1.5 to turn on a buzzer.

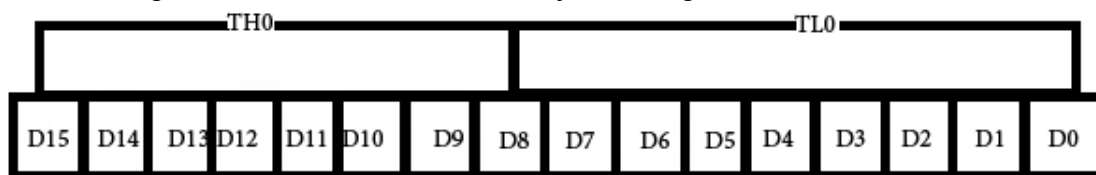
```
OVEN_HOT BIT P2.3
BUZZER BIT P1.5
HERE: JNB OVEN_HOT,HERE
      CPL BUZZER
      ACALL DELAY
      SJMP HERE
```

- ❖ A switch is connected to pin P1.7. Write a program to check the status of the switch and make the following decision. (a) If SW = 0, send "0" to P2 (b) If SW = 1, send "1" to P2

```
SW EQU P1.7
MYDATA EQU P2
HERE: MOV C,SW
      JC OVER
      MOV MYDATA,#'0'
      SJMP HERE
OVER: MOV MYDATA,#'1'
      SJMP HERE
```

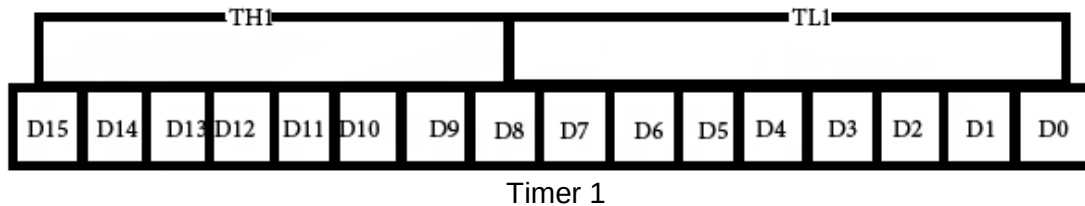
TIMERS AND COUNTERS

- ❖ The 8051 has two counters/timers which can be used either as timer to generate a time delay or as counter to count events happening outside the microcontroller.
- ❖ The 8051 has two timers: timer0 and timer1.
- ❖ They can be used either as timers or as counters.
- ❖ Both timers are 16 bits wide.
- ❖ Since the 8051 has an 8-bit architecture, each 16-bit is accessed as two separate registers of low byte and high byte.
- ❖ First we shall discuss about Timer0 registers.
- ❖ Timer0 registers is a 16 bits register and accessed as low byte and high byte.
- ❖ The low byte is referred as a TL0 and the high byte is referred as TH0.
- ❖ These registers can be accessed like any other registers.



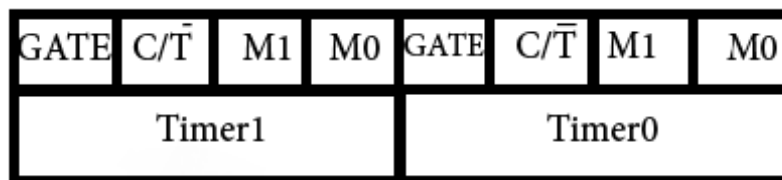
Timer 0

- ❖ Timer1 registers is also a 16 bits register and is split into two bytes, referred to as TL1 and TH1.



❖ **TMOD (timer mode) Register:**

- This is an 8-bit register which is used by both timers 0 and 1 to set the various timer modes.
- In this TMOD register, lower 4 bits are set aside for timer0 and the upper 4 bits are set aside for timer1.
- In each case, the lower 2 bits are used to set the timer mode and upper 2 bits to specify the operation.



TMOD Register

- In upper or lower 4 bits, first bit is a **GATE bit**.
- Every timer has a means of starting and stopping.
- Some timers do this by software, some by hardware, and some have both software and hardware controls.
- The hardware way of starting and stopping the timer by an external source is achieved by making **GATE=1** in the TMOD register.
- And if **GATE=0** then no need of external hardware to start and stop the timers.
- The second bit is **C/T bit** and is used to decide whether a timer is used as a **time delay generator or an event counter**.
- If this bit is **0** then it is used as a timer and if it is **1** then it is used as a counter.
- In upper or lower 4 bits, the last bits third and fourth are known as M1 and M0 respectively.
- These are used to select the timer mode.

Table 4.6: Timer Mode Operations

M1	M0	Mode	Operating Mode
0	0	0	3 bit timer mode 8 bit timer / counter THx with TLx as 5-bit prescaler
0	1	1	16 bit timer mode 16 bit timer/counters THx and TLx are cascaded; there is no prescaler
1	0	2	8 bit auto reload 8 bit auto reload timer/counter; THx holds a value that is to be reloaded into TLx each time it overflows

1	1	3	Split timer mode
---	---	---	------------------

❖ Mode 1

- It is a 16-bit timer; therefore it allows values from 0000 to FFFFH to be loaded into the timer's registers TH and TL.
- After TH and TL are loaded with a 16-bit initial value, the timer must be started.
- This is done by "SETB TR0" for timer 0 and "SETB TR1" for timer 1.
- After the timer is started.
- It starts count up until it reaches its limit of FFFFH.
- When it rolls over from FFFF to 0000H, it sets high a flag bit called TF (timer flag).
- This timer flag can be monitored. When this timer flag is raised, one option would be stop the timer with the instructions "CLR TR0" or CLR TR1 for timer 0 and timer 1 respectively.
- Again, it must be noted that each timer flag TF0 for timer 0 and TF1 for timer1.
- After the timer reaches its limit and rolls over, in order to repeat the process the registers TH and TL must be reloaded with the original value and TF must be reset to 0.
- The figure 4.11 explains the Mode 1 operation of timer

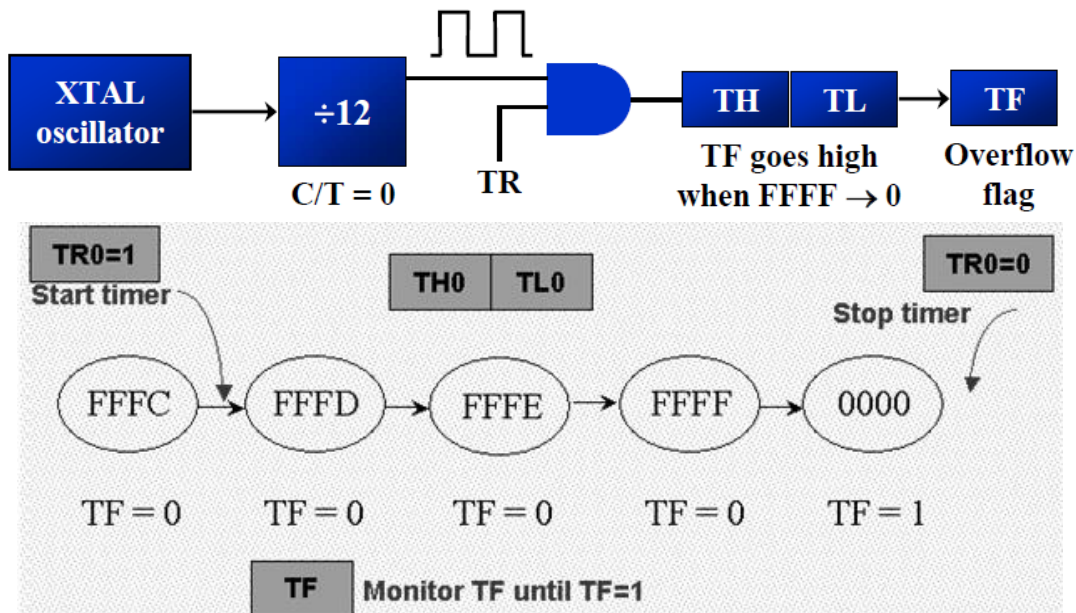


Fig 4.11 Mode 1 Operation of Timer

❖ Mode 2

- It is an 8 bit timer that allows only values of 00 to FFH to be loaded into the timer's register TH.
- After TH is loaded with 8 bit value, the 8051 gives a copy of it to TL.
- Then the timer must be started.
- It is done by the instruction "SETB TR0" for timer 0 and "SETB TR1" for timer1.

- This is like mode 1.
- After timer is started, it starts to count up by incrementing the TL register.
- It counts up until it reaches its limit of FF_H . When it rolls over from FF_H to 00. It sets high the TF (timer flag).
- If we are using timer 0, TF0 goes high; if using TF1 then TF1 is raised.
- When TL register rolls from FF_H to 00 and TF is set to 1, TL is reloaded automatically with the original value kept by the TH register.
- To repeat the process, we must simply clear TF and let it go without any need by the programmer to reload the original value.
- This makes mode 2 auto reload, in contrast in mode 1 in which programmer has to reload TH and TL.
- The figure 4.12 explains the Mode 2 operation of timer.
- The figure 4.13 explains the Mode 1 and mode 2 operation of timer with external input

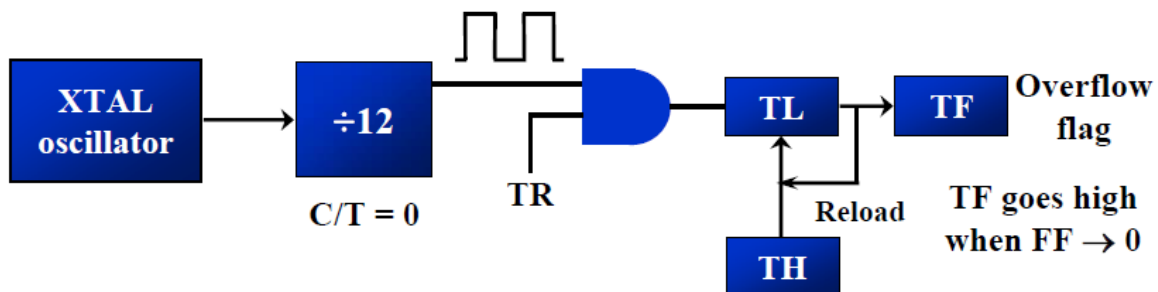
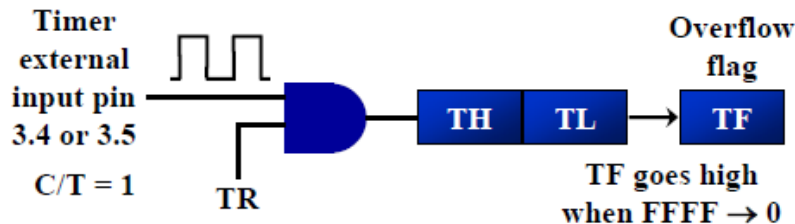


Fig 4.12 Mode 1 Operation of Timer

Timer with external input (Mode 1)



Timer with external input (Mode 2)

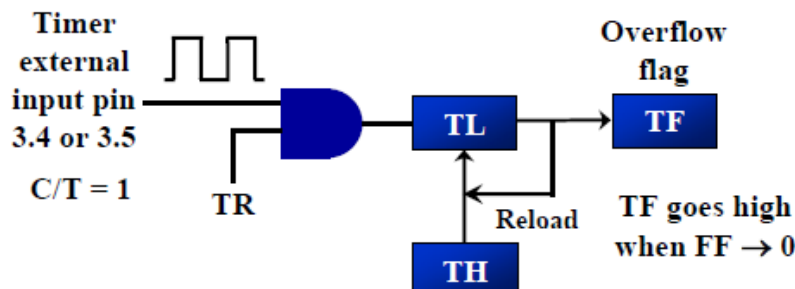


Figure 4.13 Mode 1 and Mode 2 Operation of Timer With External Input

❖ **Mode0**

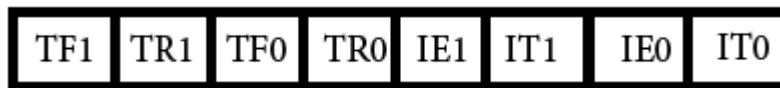
- Mode 0 is exactly same like mode 1 except that it is a 13-bit timer instead of 16-bit.
- The 13- bit counter can hold values between 0000 to 1FFFH in TH-TL.
- Therefore, when the timer reaches its maximum of 1FFFH, it rolls over to 0000, and TF is raised.

❖ **Mode3**

- Mode 3 is also known as a split timer mode.
- Timer 0 and 1 may be programmed to be in mode 0, 1 and 2 independently of similar mode for other timer.
- This is not true for mode 3; timers do not operate independently if mode 3 is chosen for timer 0.
- Placing timer 1 in mode 3 causes it to stop counting; the control bit TR1 and the timer 1 flag TF1 are then used by timer0.

❖ **TCON register**

- Bits and symbol and functions of every bits of TCON are as follows:



TCON Register

Table 4.7: TCON Register Description

BIT	Symbol	Functions
7	TF1	Timer1 over flow flag. Set when timer rolls from all 1s to 0. Cleared When the processor vectors to execute interrupt service routine Located at program address 001Bh.
6	TR1	Timer 1 run control bit. Set to 1 by programmer to enable timer to count; Cleared to 0 by program to halt timer.
5	TF0	Timer 0 over flow flag. Same as TF1.
4	TR0	Timer 0 run control bit. Same as TR1.
3	IE1	External interrupt 1 Edge flag. Not related to timer operations
2	IT1	External interrupt1 signal type control bit. Set to 1 by program to Enable external interrupt 1 to be triggered by a falling edge signal. Set To 0 by program to enable a low level signal on external interrupt1 to generate an interrupt.
1	IE0	External interrupt 0 Edge flag. Not related to timer operations.
0	IT0	External interrupt 0 signal type control bit. Same as IT0

Calculating Time Delay: (By Assuming XTAL Frequency as 11.0592MHz)

In Hex	In Decimal
$(FFFF - YYXX + 1) * 1.085\mu s$ Where YYXX are TH, TL initial values respectively.	Convert YYXX values of the TH, TL registers to decimal to get a NNNNN decimal number, then $(65536 - NNNNN) * 1.085 \mu s$

Finding the values to be loaded into the timer:

- ❖ Assuming XTAL = 11.0592MHz
 1. Divide the desired time delay by 1.085us
 2. Perform $65536 - n$, where n is the decimal value we got from step 1
 3. Convert the result of step 2 to hex, where YYXX is the initial hex value to be loaded into the timer's registers
 4. Set TL =xx and TH =yy

Example

- ❖ Indicate which mode and which timer are selected for each of the following.
(a) MOV TMOD, #01H (b) MOV TMOD, #20H (c) MOV TMOD, #12H

Solution:

We convert the value from hex to binary. From Figure 9-3 we have:

- (a) TMOD = 00000001, mode 1 of timer 0 is selected.
- (b) TMOD = 00100000, mode 2 of timer 1 is selected.
- (c) TMOD = 00010010, mode 2 of timer 0, and mode 1 of timer 1 are

- ❖ Find the timer's clock frequency and its period for various 8051-based system, with the crystal frequency 11.0592 MHz when C/T bit of TMOD is 0.

Solution:

$$(1/12) \times 11.0529 \text{ MHz} = 921.6 \text{ kHz};$$

$$T = 1/921.6 \text{ kHz} = 1.085 \mu s \text{ selected.}$$

- ❖ Write a program to create square wave of 50% duty cycle (with equal portions high and low) on the P1.5 bit use Timer 0 to generate the time delay.

Solution:

```
MOV TMOD,#01      ;Timer 0, mode 1(16-bit mode)
HERE: MOV TL0,#0F2H ;TL0=F2H, the low byte
      MOV TH0,#0FFH ;TH0=FFH, the high byte
      CPL P1.5      ;toggle P1.5
      ACALL DELAY
      SJMP HERE
DELAY:
      SETB TR0      ;start the timer 0
AGAIN: JNB TF0,AGAIN ;monitor timer flag 0
      ;until it rolls over
      CLR TR0       ;stop timer 0
      CLR TF0       ;clear timer 0 flag
      RET
```

- ❖ **Write a program to continuously generate a square wave of 2 kHz frequency on pin P1.5 using timer 1. Assume the crystal oscillator frequency to be 12 MHz.**

The period of the square wave is $T = 1/(2 \text{ kHz}) = 500 \text{ us}$.

Each half pulse = 250 us.

The value n for 250 us is: $250 \text{ us} / 1 \text{ us} = 250$

$65536 - 250 = \text{FF06H}$.

TL = 06H and TH = 0FFH.

```

MOV TMOD,#10          ;Timer 1, mode 1
AGAIN: MOV TL1,#06H    ;TL0 = 06H
      MOV TH1,#0FFH    ;TH0 = FFH
      SETB TR1         ;Start timer 1
BACK:  JNB TF1,BACK     ;Stay until timer rolls over
      CLR TR1         ;Stop timer 1
      CPL P1.5        ;Complement P1.5 to get Hi, Lo
      CLR TF1         ;Clear timer flag 1
      SJMP AGAIN      ;Reload timer

```

- ❖ **Write a program segment that uses timer 1 in mode 2 to toggle P1.0 once whenever the counter reaches a count of 100. Assume the timer clock is taken from external source P3.5 (T1).**

The TMOD value is 60H

The initialization value to be loaded into TH1 is

$256 - 100 = 156 = 9\text{CH}$

```

MOV TMOD,#60h        ;Counter1, mode 2, C/T'= 1
MOV TH1,#9Ch         ;Counting 100 pulses
SETB P3.5            ;Make T1 input
SETB TR1             ;Start timer 1
BACK: JNB TF1,BACK    ;Keep doing it if TF = 0
      CPL P1.0        ;Toggle port bit
      CLR TF1         ;Clear timer overflow flag
      SJMP BACK       ;Keep doing it

```

SERIAL COMMUNICATION:

Baud Rate:

- ❖ The 8051 transfers and receives data serially at many different baud rates.
- ❖ The baud rate in the 8051 is programmable.
- ❖ This is done with the help of Timer 1.
- ❖ The relationship between the crystal frequency and the baud rate is
 - For XTAL = 11.0592 MHz, the machine cycle frequency is 921.6kHz
 - The UART circuitry divides the machine cycle frequency of 921.6kHz by 32 once more before it is used by Timer 1 to set the baud rate. Therefore it gives 28,800Hz.
 - When Timer 1 is used to set the baud rate it must be programmed in mode 2, i.e., 8-bit, auto reload.

- To get baud rates compatible with the PC, TH1 should be loaded with the values shown in the following table by assuming XTAL=11.0592MHz.

Table 4.8: Baud Rate Value For TH1

Baud Rate	TH1 (Decimal)	TH1 (Hex)
9600	-3	FD
4800	-6	FA
2400	-12	F4
1200	-24	E8

SBUF Register

- ❖ SBUF is an 8-bit register used solely for serial communication in the 8051.
- ❖ For a byte of data to be transferred via the TxD line, it must be placed in the SBUF register.
- ❖ Similarly, SBUF holds the byte of data when it is received by the RxD line.
- ❖ SBUF can be accessed like any other registers in the 8051
MOV SBUF, #'D'
MOV SBUF, A
MOV A, SBUF
- ❖ The moment a byte is written into SBUF, it is framed with the start and stop bits and transferred serially via the TxD pin
- ❖ Similarly when the bits are received serially via RxD, the 8051 deframes it by eliminating the start and stop bits, making a byte out of the data received, and then placing it in the SBUF

SCON Register:

- ❖ The SCON register is an 8-bit register used to program the start bit, stop bit and data bits of data framing, among other things.

SM0	SM1	SM2	REN	TB8	RB8	TI	RI
-----	-----	-----	-----	-----	-----	----	----

SM0	SCON.7	Serial port mode specifier
SM1	SCON.6	Serial port mode specifier
SM2	SCON.5	Used for multiprocessor communication. (Make it 0)
REN	SCON.4	Set/cleared by software to enable/disable reception.
TB8	SCON.3	Not widely used.
RB8	SCON.2	Not widely used.
TI	SCON.1	Transmit interrupt flag. Set by hardware at the beginning of the stop bit in mode 1. Must be cleared by software.
RI	SCON.0	Receive interrupt flag. Set by hardware halfway through the stop bit time in mode 1. Must be cleared by software.

Note: Make SM2, TB8, and RB8 = 0.

❖ SM0, SM1:

- These two bits determines the framing of data by the specifying the number of bits per character, and the start and stop bits.
- They take the following combinations

Table 4.9: Serial Mode Operation

SM0	SM1	
0	0	Serial Mode 0
0	1	Serial Mode 1, 8-bit data, 1 start bit, 1 stop bit
1	0	Serial Mode 2
1	1	Serial Mode 3

- Serial Mode 0, 2 and 3 are not commonly used nowadays.
- Serial Mode 1 is compatible with the COM port of IBM/PC's
- Serial Mode 1, allows the baud rate to be variable and is set by Timer 1 of the 8051
- In Serial Mode 1, for each character a total of 10 bits are transferred
- ❖ **SM2**
 - This bit enables the multiprocessing capability of the 8051 by assigning 'SM2=1'.
 - If 'SM2=0' multiprocessing capability is disabled.
- ❖ **REN: (Receiver Enable)**
 - When the REN bit is high, it allows the 8051 to receive data on the RxD pin
 - If we want the 8051 to bit transfer and receive the data, REN must be set to 1
 - By making REN=0, the receiver is disabled
 - Instruction to set and Clear REN is
SETB SCON.4
CLR SCON.4
- ❖ **TB8: (Transfer Bit 8)**
 - It is used for serial mode 2 and 3.
 - If this is not used assign TB8 as 0
- ❖ **RB8 (Receive Bit 8)**
 - In serial mode 1, this bit gets a copy of the stop bit when 8-bit data is received.
 - This bit is rarely used anymore, it is assigned as 0
- ❖ **TI (Transmit Interrupt)**
 - When 8051 finishes the transfer of the 8-bit character, it raises the TI flag to indicate that it is ready to transfer another byte.
 - The TI bit is raised at the beginning of the stop bit.
- ❖ **RI (Receive Interrupt)**
 - When the 8051 receives data serially via RxD, it gets rid of the start and stop bits and places the byte in the SBUF register.
 - Then it raises the RI flag bit to indicate that a byte has been received and should be picked up before it is lost.
 - RI is raised halfway through the stop bit.

Program to Transfer Data Serially:

1. The TMOD register is loaded with the value 20H, indicating the use of Timer 1 in mode 2 to set baud rate
2. The TH1 is loaded with the value to set the baud rate for serial data transfer
3. The SCON register is loaded with the value 50H, indicating Serial Mode 1
4. TR1 is set to start Timer 1
5. TI is cleared by the 'CLR TI' instruction
6. The character byte to be transferred serially is written into the SBUF register
7. The TI flag bit is monitored with the instruction 'JNB TI, XX'
8. To Transfer next character goto Step 5

Program to Receive Data Serially:

1. The TMOD register is loaded with the value 20H, indicating the use of Timer 1 in mode 2 to set baud rate
2. The TH1 is loaded with the value to set the baud rate for serial data transfer
3. The SCON register is loaded with the value 50H, indicating Serial Mode 1 and receive enable is turned ON
4. TR1 is set to start Timer 1
5. RI is cleared by the 'CLR RI' instruction
6. The RI flag bit is monitored with the instruction 'JNB RI, XX'
7. When RI is raised, SBUF has the byte
8. To Receive next character goto Step 5

Doubling the Baud Rate:

- ❖ Use a higher frequency crystal
- ❖ Change bit in the PCON register, shown below

PCON Register

7	6	5	4	3	2	1	0
SMOD	-	-	-	GF1	GF0	PD	IDL

- ❖ Baud Rate Calculation for SMOD (Serial Mode) = 0
Machine Cycle Frequency = $\frac{\text{Crystal Frequency}}{12 \times 32}$
- ❖ Baud Rate Calculation for SMOD (Serial Mode) = 1
Machine Cycle Frequency = $\frac{\text{Crystal Frequency}}{12 \times 16}$

Table 4.10: Baud Rate Comparison For SMOD=0 Vs SMOD=1

TH1 (Decimal)	TH1 (Hex)	SMOD = 0	SMOD = 1
-3	FD	9600	19200
-6	FA	4800	9600
-12	F4	2400	4800
-24	E8	1200	2400

Example

- ❖ Write a program to transfer a letter 'Y' serially at 9600 baud continuously, and also to send a letter 'N' through Port 0, which is connected to a display device.

```
MOV TMOD,#20H
MOV TH1,#-3
MOV SCON, #50H
SETB TR1
YY: MOV SBUF,#'Y'
XX: JNB TI, XX
    CLR TI
    MOV P0, #'N'
    SJMP YY
```

- ❖ Write a program to receive the data which has been sent in serial form and send it out to port 0 in parallel form. Also save the data at RAM location 60H.

```
MOV TMOD,#20H
MOV TH1,#-3
```



```

MOV SCON, #50H
SETB TR1
CLR RI
XX:  JNB RI, XX
      MOV A, SBUF
      MOV P0, A
      MOV 60H, A

```

INTERRUPT

Interrupts Vs Polling:

- ❖ A single microcontroller can serve several devices. There are two ways to do that:
 - Interrupts
 - Polling
- ❖ In the interrupt method, whenever any device needs its service, the device notifies the microcontroller by sending it an interrupt signal.
- ❖ Upon receiving an interrupt signal, the microcontroller interrupts whatever it is doing and serves the device.
- ❖ The program associated with the interrupt is called the interrupt service routine (ISR) or interrupt handler.
- ❖ In polling, the microcontroller continuously monitors the status of a given device; when the status condition is met, it performs the service.
- ❖ After that it moves on to monitor the next device until each one is serviced.
- ❖ Although polling can monitor the status of several devices and serve each of them as certain conditions are met, it is not an efficient use of the microcontroller.
- ❖ The advantage of interrupts is that the microcontroller can serve many devices; each device can get the attention of the microcontroller based on the priority assigned to it.
- ❖ The polling method cannot assign priority since it checks all devices in a round-robin fashion.
- ❖ In interrupt method, interrupts can be masked which is not possible in polling method.
- ❖ Polling method wastes microcontroller's time by polling devices.

Interrupt Service Routine:

- ❖ For every interrupt there must be an ISR or Interrupt Handler
- ❖ When an interrupt is invoked, the microcontroller runs the ISR.
- ❖ For every interrupt, there is a fixed location in memory that holds the address of its ISR
- ❖ The group of memory locations set aside to hold the addresses of ISRs is called the interrupt vector table, shown in table 4.12.

Steps in executing an interrupt

Upon activation of an interrupt, the microcontroller goes through the following steps.

1. It finishes the instruction it is executing and saves the address of the next instruction (PC) on the stack
2. It also saves the current status of all interrupts internally
3. It jumps to a fixed location in memory called the interrupt vector table that holds the address of the interrupt service routine

4. The microcontroller gets the address of the ISR from the interrupt vector table and jumps to it. It starts to execute the interrupt service subroutine until it reaches the last instruction of the subroutine, which is RETI
5. Upon executing the RETI instruction, the microcontroller returns to the place where it was interrupted. First it gets the PC address from the stack by popping the two bytes of the stack into PC. Then it starts to execute from the address.

Interrupt Sources

- ❖ The 8051 architecture can handle interrupts from 5 sources.
- ❖ These are:
 - Two external interrupt lines,
 - Two timers
 - Serial interface
- ❖ Each one of these is assigned an interrupt vector address. This is quite similar to the RST interrupt vectors in the case of 8085.
- ❖ **External Interrupts**
 - Port P3 of 8051 is a multi-function port.
 - Different lines of this port carry out functions which are additional to data input-output on the port.
 - Lines P3.2 and P3.3 can be used as interrupt inputs.
 - Interrupts will be caused by a 'LOW' level, or a negative edge on these lines.
 - Half of the special function register TCON is used for setting the conditions for causing interrupts from external sources. This register is bit addressable.

SFR TCON at byte address 88H



Table 4.11: TCON Register Description

BIT	Symbol	Functions
3	IE1	External interrupt 1 Edge flag. Not related to timer operations
2	IT1	External interrupt1 signal type control bit. Set to 1 by program to Enable external interrupt 1 to be triggered by a falling edge signal. (Edge Sensitive) Set to 0 by program to enable a low level signal on external interrupt 1 to generate an interrupt. (Level Sensitive)
1	IE0	External interrupt 0 Edge flag. Not related to timer operations.
0	IT0	External interrupt 0 signal type control bit. Set to 1 by program to Enable external interrupt 0 to be triggered by a falling edge signal. (Edge Sensitive) Set to 0 by program to enable a low level signal on external interrupt 0 to generate an interrupt. (Level Sensitive)

- If the flag is 1, the selected type of event (edge or level) has occurred on the corresponding interrupt line.

❖ Internal Interrupts

- Internally generated interrupts can be from either timer, or from the serial interface.
- The serial interface causes interrupts due to a receive event (RI) or due to a transmit event (TI).
- The receive event occurs when the input buffer of the serial line (SBUF in) is full and a byte needs to be read from it.
- The transmit event indicates that a byte has been sent a new byte can be written to output buffer of the serial line (SBUF out).
- 8051 timers always count up. When their count rolls over from the maximum count to 0000, they set the corresponding timer flag TF1 or TF0 in TCON.
- Counters run only while their run flag (TR1 or TR0) is set by the user program.
- When the run flag is cleared, the count stops incrementing.
- The 8051 can be setup so that an interrupt occurs whenever TF1 or TF0 is set.

❖ Enabling Interrupts

- At power-up, all interrupts are disabled.
- Suppose Timer 0 is started. When it times out, TF0 in the special function register TCON will be set.
- However, this will not cause an interrupt.
- To enable interrupts, a number of steps need to be taken.
- Interrupts are enabled in a manner which is quite similar to the 8085.
- There is an interrupt enable special function register IE at byte address A8H.
- This register is bit addressable. (The assembler gives special mnemonics to each bit address.)

SFR Interrupt Enable (IE) Register

EA	--	ET2	ES	ET1	EX1	ET0	EX0
----	----	-----	----	-----	-----	-----	-----

EA IE.7 Disables all interrupts.

If **EA=0**, no interrupt is acknowledged.

If **EA=1**, each interrupt source is individually enabled or disabled by setting or clearing its enable bit.

--- IE.6 Not implemented, reserved for future use. *

ET2 IE.5 Enables or disables timer 2 overflow or capture interrupt (8052).

ES IE.4 Enables or disables the serial port interrupt. **RI or TI**

ET1 IE.3 Enables or disables timer 1 overflow interrupt. **TF1**

EX1 IE.2 enables or disables external interrupt 1. **INT1**

ET0 IE.1 Enables or disables timer 0 overflow interrupt. **TF0**

EX0 IE.0 enables or disables external interrupt 0. **INT0**



❖ Interrupt Vectors

- When an interrupt occurs, the updated PC is pushed on the stack and is loaded with the vector address corresponding to the interrupt.
- The following table 4.12 gives the vector addresses.
- The order of entries in the table is also the order in which the 8051 will poll these in case of multiple interrupts

Table 4.12: Interrupt and Vector Address

Interrupt Source	Vector Address
External Interrupt 0	0003H
Timer 0 Overflow	000BH
External Interrupt 1	0013H
Timer 1 Overflow	001BH
Serial Interface	0023H

- 8051 starts executing from address 0000H at power-up or reset.
- The first 3 bytes are typically used for placing a long jump instruction to start of the code area.
- The interrupt vectors start from 0003 and are separated by 8 bytes from each other.
- Many simple interrupt handlers can be accommodated in this space.
- Otherwise, jump instructions (to handler locations) need to be placed at the vector addresses.
- This is quite similar to the RST handlers for 8085.
- Thus, to enable interrupts from T0, we have to do
SetB EA ;(or SetB IE.7) to enable interrupts
SetB ET0 ;(or SetB IE.1) to enable interrupts from T0
- After this, whenever T0 overflows, TF0 will be set (in SFR TCON), the currently running program will be interrupted, its PC value will be put on the stack (PC-L first, PC-H after – because the stack grows upwards in 8051), and PC will be loaded with 000B H.
- The interrupt handler for T0 should be placed here, and it should end with the instruction: RETI

❖ Interrupt Priorities

- 8051 has two levels of interrupt priorities:
 - High or Low.
- When 8051 is powered up, the priorities are assigned according to the table 4.13.

Table 4.13 8051/52 Interrupt Priority Upon Reset

Highest to Lowest Priority	
External Interrupt 0	INT0
Timer Interrupt 0	TF0
External Interrupt 1	INT1

Timer Interrupt 1	TF1
Serial Communication	RI + TI
Timer 2 (8052 only)	TF2

○ **Setting Interrupt Priority**

- By assigning priorities, the order of multiple interrupts can be controlled and serviced.
- Priorities are set by bits in a special function register called IP (Interrupt Priority), which is at the byte address B8H.
- This register is also bit addressable.
- The assembler defines special names for bits of this register.

SFR IP Register at byte address B8H

--	--	PT2	PS	PT1	PX1	PT0	PX0
----	----	-----	----	-----	-----	-----	-----

Priority bit = 1 assigns high priority. Priority bit = 0 assigns low priority.

--	IP.7	Reserved
--	IP.6	Reserved
PT2	IP.5	Timer 2 interrupt priority bit (8052 only)
PS	IP.4	Serial port interrupt priority bit
PT1	IP.3	Timer 1 interrupt priority bit
PX1	IP.2	External interrupt 1 priority bit
PT0	IP.1	Timer 0 interrupt priority bit
PX0	IP.0	External interrupt 0 priority bit

User software should never write 1s to unimplemented bits, since they may be used in future products.

- ❖ Notice that the bits are in the polling order of interrupts.
- ❖ A '1' in a bit position assigns a high priority to the corresponding source of interrupts and '0' gives it a low priority.
- ❖ In case of multiple interrupts, the following rules apply:
 - While a low priority interrupt handler is running, if a high priority interrupt arrives, the handler will be interrupted and the high priority handler will run. When the high priority handler does 'RETI', the low priority handler will resume. When this handler does 'RETI', control is passed back to the main program.
 - If a high priority interrupt is running, it cannot be interrupted by any other source – even if it is a high priority interrupt which is higher in polling order.
 - A low-priority interrupt handler will be invoked only if no other interrupt is already executing. Again, the low priority interrupt cannot preempt another low priority interrupt, even if the later one is higher in polling order.
 - If two interrupts occur at the same time, the interrupt with higher priority will execute first. If both interrupts are of the same priority, the interrupt which is

higher in polling sequence will be executed first. This is the only context in which the polling sequence matters.

Serial Interrupts

- ❖ Serial interrupts are handled somewhat differently from the timers.
- ❖ There are independent interrupt flags for reception and transmission of serial data, called RI and TI.
- ❖ RI indicates that a byte has been received and is available for reading in the input buffer.
- ❖ TI indicates that the previous byte has been sent serially and a new byte can be written to the serial port.
- ❖ A serial interrupt occurs if either of these flags is set. (Of course the serial interrupt must be enabled for this to occur).
- ❖ The interrupt service routine should check which of these events caused the interrupt.
- ❖ This can be done by examining the flags.
- ❖ Either or both of the flag might be set, requiring a read from or write to the serial buffer SBUF (or both).
- ❖ Recall that the input and output buffers are distinct but are located at the same address.
- ❖ A read from this address reads the input buffer while a write to the same address writes to the output buffer.
- ❖ The RI and TI flags are not automatically cleared when an interrupt is serviced.
- ❖ Therefore, the interrupt service routine must clear them before returning.
- ❖ Here is an example handler for serial interrupts:
- ❖ **Write a program in which the 8051 reads data from P1 and writes it to P2 continuously while giving a copy of it to the serial COM port to be transferred serially. Assume that XTAL=11.0592MHz. Set the baud rate at 9600.**

```
                ORG 0
                LJMP MAIN
                ORG 23H
                LJMP SERIAL

MAIN:           ORG 30H
                MOV P1,#0FFH
                MOV TMOD,#20H
                MOV TH1, #0FDH
                MOV SCON,#50H
                MOV IE,#10010000B
                SETB TR1
BACK:           MOV A,P1
                MOV SBUF, A
                MOV P2,A
                SJMP BACK

SERIAL:         ORG 100
                JB TI, TRANS
                MOV A, SBUF
                CLR RI
```

```

TRANS:    RETI
          CLR TI
          RETI

```

- ❖ Write a program in which 10 bytes of data stored in RAM locations starting from 45H are transferred serially. At the end of data transfer, the value of R0 is displayed on P1. Assume that XTAL=11.0592MHz. Set the baud rate at 9600.

```

          ORG 0
          LJMP MAIN
          ORG 23H
          LJMP SERIAL

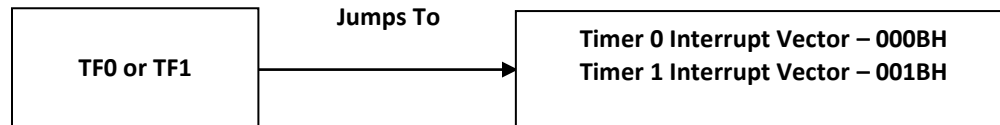
MAIN:     ORG 30H
          MOV TMOD,#20H
          MOV TH1, #0FDH
          MOV SCON,#50H
          MOV IE,#10010000B
          SETB TR1
          MOV R0,#10
          MOV R1,#45H
BACK:     MOV A,@R1
          MOV SBUF, A
          DJNZ R0, BACK
HERE:     SJMP HERE

          ORG 100
SERIAL:   JNB TI, RECE
          MOV A,R0
          MOV P1,A
          CLR TI
          RETI
RECE:     MOV A, SBUF
          CLR RI
          RETI

```

Programming Timer Interrupts

- ❖ The timer interrupts IT0 and IT1 are related to Timers 0 and 1, respectively.
- ❖ If the timer interrupt in the IE register is enabled, whenever the timer rolls over, TF is raised, and the microcontroller is interrupted in whatever it is doing, and jumps to the interrupt vector table to service the ISR.
- ❖ In this way, the microcontroller can do other things until it is notified that the timer has rolled over, Refer figure . can do other
- ❖ We must avoid using the memory space allocated to the interrupt vector table. Therefore, we place all the initialization codes in memory starting at 30H. The LJMP instruction is the first instruction that the 8051 first executes when it is powered up. LJMP redirects the controller away from the interrupt vector table.



- ❖ The interrupt programming for timers involves following steps :
 1. Configure TMOD register to select timer(s) and its/their mode.
 2. Load initial values in THx and TLx for mode 0 and 1; or in THx only for mode 2.
 3. Enable Timer Interrupt by configuring bits of IE register.
 4. Start timer by setting timer run bit TRx.
 5. Write subroutine for Timer Interrupt. The interrupt number is 1 for Timer 0 and 3 for Timer1.

Note that it is not required to clear timer flag TFX.

6. To stop the timer, clear TRx in the end of subroutine. Otherwise it will restart from 0000H in case of modes 0 or 1 and from initial values in case of mode 2.
7. If the Timer has to run again and again, it is required to reload initial values within the routine itself (in case of mode 0 and 1). Otherwise after one cycle timer will start counting from 0000H.

Example:

- ❖ Write a program that displays a value of 'Y' at port 0 and 'N' at port 2 and also generates a square wave at 10kHz, with Timer 0 in mode 2 at port pin P1.2. XTAL=22MHz.

```

                ORG 0000H
                LJMP MAIN

                ORG 000B
                CPL 1.2
                RETI

MAIN:           ORG 0030H
                MOV TMOD,#02H
                MOV TH0,#0B6H
                MOV IE,#82H
                SETB TR0
BACK:          MOV P0,#'Y'
                MOV P2,#'N'
                SJMP BACK
                END
  
```

- ❖ Write a ALP to generates a wave form with high portion of 1085uS and a low portion of 15us. Also acquire the value from Port 0 continuously and move it to Port 1. Assume XTAL=11.059MHz. Use Timer 1.

```

                ORG 0000H
                LJMP MAIN
  
```



```

                ORG 000B
                LMP ISR_T1

                ORG 0030H
MAIN:          MOV TMOD,#10H
                MOV TH1,#0FCH
                MOV TL1,#018H
                MOV P0,#0FFH
                MOV IE,#88H
                SETB TR1
BACK:          MOV A, P0
                MOV P1,A
                SJMP BACK

ISR_T1         CLR TR1
                CLR P2.1
                MOV R2,#04
HERE:          DJNZ R2,HERE      ; 8 M/C (2 Machine Cycle * 4 Count)
                MOV TL1,#18H    ; 2 M/C
                MOV TH1,#0FCH   ; 2 M/C
                SETB TR1        ; 1 M/C
                SETB P2.1       ; 1 M/C
                RETI             ; Total = 14M/C *1.085uS =15.19uS
                END

```

Programming External Interrupts

- ❖ The external interrupts are the interrupts received from the (external) devices interfaced with the microcontroller.
- ❖ They are received at INTx pins of the controller.
- ❖ These can be level triggered or edge triggered.
- ❖ In level triggered, interrupt is enabled for a low at INTx pin; while in case of edge triggering, interrupt is enabled for a high to low transition at INTx pin.
- ❖ The edge or level trigger is decided by the TCON register.
- ❖ Note : For a level trigger interrupt, the INTx pin must remain low until the start of the ISR and should return to high before the end of ISR. If the low at INTx pin goes high before the start of ISR, interrupt will not be generated. Also if the INTx pin remains low even after the end of ISR, the interrupt will be generated once again. This is the reason why level trigger interrupt (low) at INTx pin must be four machine cycles long and not greater than or smaller than this.
- ❖ Following are the steps for using external interrupt :
 - Enable external interrupt by configuring IE register.
 - Write routine for external interrupt. The interrupt number is 0 for EX0 and 2 for EX1 respectively.

Example:

- ❖ Two switches are connected to pins P3.2 and P3.3. When a switch is pressed, the corresponding line goes low. Write a program to
 - Light all LEDs connected to port 0, if the first switch is pressed
 - Light all LEDs connected to port 2, if the second switch is pressed

```
                ORG 0000
                LJMP MAIN

LED1:           ORG 0003
                MOV P0,#0FF
                MOV R0,#255
                DJNZ R0,LED1
                RETI

LED2:           ORG 0013
                MOV P2,#0FF
                MOV R2,#255
                DJNZ R2,LED2
                RETI

MAIN:           ORG 0030
                MOV IE,#85H
HERE:           SJMP HERE
                END
```

- ❖ Generate from all pins of Port 0, a Square wave which is half the frequency of the signal applied at INT0 pin.

```
                ORG 0000
                LJMP MAIN

                ORG 0003
                CPL P0
                RETI

MAIN:           ORG 0030
                SETB TCON.0      ; EDGE TRIGGERED
                MOV IE,#81
HERE:           SJMP HERE
```